

Multi class SVM algorithm with active learning for network traffic classification

Shi Dong^{a,b,*}

^a School of Computer Science and Technology, Zhoukou Normal University, Zhoukou 466001, China

^b State Key Laboratory of Networking and Switching Technology (Beijing University of Posts and Telecommunications), Beijing 100876, China

ARTICLE INFO

Keywords:

Traffic classification
NETFLOW flow
Imbalance problem
Machine learning
SVM
CMSVM

ABSTRACT

With the current massive amount of traffic that is going through the internet, internet service providers (ISPs) and networking service providers (NSPs) are looking for various ways to accurately predict the application type of flow that is going through the internet. Such prediction is critical for security and network monitoring applications as they require application type to be known in prior. Traditional ways using port-based or payload-based analysis are not sufficient anymore as many applications start using dynamic unknown port numbers, masquerading, and encryption techniques to avoid being detected. Recently, machine learning has gained significant attention in many prediction applications including traffic classification from flow features or characteristics. However, such algorithms suffer from an imbalanced data problem where some applications have fewer flow data and hence difficult to predict. In this paper, we employ network flow-level characteristics to identify the application type of traffic. Furthermore, we propose the use of an improved support vector machine (SVM) algorithm, named cost-sensitive SVM (CMSVM), to solve the imbalance problem in network traffic identification. CMSVM adopts a multi-class SVM algorithm with active learning which dynamically assigns a weight for applications. We examine the classification accuracy and performance of the CMSVM algorithm using two different datasets, namely MOORE_SET and NOC_SET datasets. Our results show that the CMSVM algorithm can reduce computation cost, improve classification accuracy and solve the imbalance problem when compared to other machine learning techniques.

1. Introduction

Network traffic classification is a fundamental issue for various network operations including security and network management. One fundamental issue in that field is to classify applications or know their type from their network flow behavior. This is particularly important to internet service providers (ISPs) and networking service providers (NSPs) where a vast amount of traffic passes with different applications that have different requirements. In such large networks, it is critical to distinguish real-time traffic applications and give them a higher priority than those non-real-time applications. In addition to prioritization, application classification can help in designing and engineering the network in terms of workload characterization, capacity planning, and routing provisioning for future application needs. Moreover, such classification can help detect and analyze malicious traffic that affects the network which follows specific behavior that is known to be malicious but can not be detected by simply looking at the endpoints. Looking at

the flow of the application and its network behavior can detect and possibly prevent such applications from affecting the network. Thus, knowing the type of applications has been a major concern in both research and industry.

Traditionally, port-based and payload based approach has been used to know the type of application by knowing its payload or port number. Applications in such approaches are mapped to specific well known and defined ports and hence can be detected by looking at the port number only. However, applications such as point to point (P2P) and ad hoc do not adopt specific port and replace that dynamic unknown ports. Hence, only 50–60% of applications were correctly classified using port-based approaches, as was reported in 2005 Moore and Papagiannaki (2005). In addition to port-based, deep packet inspection (DPI) techniques are also used to classify applications in previous. However, such techniques fail to identify the encrypted network traffic, which becomes a new challenge for traffic identification Kumar, Turner, and Williams (2006). Hence, the use of such traditional techniques is not sufficient enough to

* Address: School of Computer Science and Technology, Zhoukou Normal University, Zhoukou 466001, China.

E-mail address: dongshi@zknpu.edu.cn.

<https://doi.org/10.1016/j.eswa.2021.114885>

Received 16 October 2019; Received in revised form 11 February 2021; Accepted 6 March 2021

Available online 11 March 2021

0957-4174/© 2021 Elsevier Ltd. All rights reserved.

classify applications that are going in today's network.

Nowadays, advances in machine learning have proven their efficiency in prediction applications, especially in the networking field. Learning-based techniques can prove efficiency in the classification of various applications if they were trained to count for a large amount of ongoing and complex applications using sufficient datasets. It uses different probability theories and statistical-based behaviors to identify the network traffic application with sufficient time, cost, and complexity. Montieri, Ciunzo, Aceto, and Pescapé (2017) study on whether anonymity networks (such as Tor, JonDonym, and I2P) can be discerned. Experimental results show that the considered anonymity networks (Tor, I2P, JonDonym) can be easily identified. Giuseppe Aceto, Ciunzo, Montieri, and Pescapé (2018), Aceto, Ciunzo, Montieri, Pescapé (2019a), Aceto, Ciunzo, Montieri, Pescapé (2019b) proposed a series of multi-model fusion algorithms for mobile encrypted traffic. Experimental results indicate proposed algorithms can effectively improve the identification performance. Hence, lots of machine learning algorithms have been used to classify applications in various networks. Examples of that can be found in surveys such as Nguyen and Armitage (2009), Dainotti, Pescape, and Claffy (2012). Despite that fact, a common burden in such approaches is the imbalance problem of some applications in the datasets. Datasets are normally collected from the network and hence some applications have only a limited number of samples which made it difficult for learning algorithms to learn and classify such applications Buda, Maki, and Mazurowski (2018). Such a problem sound to be persistent with most network datasets, except datasets that were actually designed to have equally distributed samples of different application traffics which makes them inapplicable and hard to build datasets. Hence, the imbalance problem has become a challenge for traffic identification based on machine learning in the actual network application traffics.

In this work, we propose an algorithm to solve the imbalance problem using imbalanced datasets and a modified version of the support vector machine (SVM) learning algorithm. In other words, we extend the work done in Shi, Zhou, Zhou, Wei, and Jian (2013) is proposing the use of another learning algorithm that enhanced classification accuracy with the reduction of cost and complexity. First, we present the new cost-sensitive SVM (CMSVM) learning algorithm and compare it with the traditional SVM. CMSVM uses multi-class SVM with active learning algorithms to resolve the imbalance problem for different applications by adaptively learning weights. We applied the proposed algorithm to two existing datasets, namely MOORE_SET Moore and Papagiannaki (2005), and NOC_SET Shi, Zhou, Wei, and Jian (2013), datasets. Then, we verified the performance of the proposed algorithm in terms of accuracy, precision, recall, and effectiveness in solving the imbalance problem. We compare our performance to other highly used machine learning techniques to prove our efficiency in outperforming others and solving the imbalance problem with the used dataset.

The rest of the paper is organized as follows: Section 2 provides the related work about machine learning algorithms and traffic identification models on NETFLOW sampled flow records. Section 3 discussed the problem model and propose CMSVM. Section 4 highlights the datasets used later for the evaluation of the proposed model. Section 5 describes an evaluation method and analyzes experiment results. Finally, conclusions are drawn in Section 6.

2. Background

In this section, we present the state of the art in the domain of machine learning techniques for network traffic classification and application identification. We highlight our contribution in comparison to the prior work. In addition, we discuss some of the primary definitions that are important to understand the later machine learning approaches.

2.1. Related works

Traffic identification using supervised machine learning techniques started earlier in 2005 when Naïve and Kernel Bayesian classifiers were used to classify internet traffic with different applications Moore and Zuev (2005). Such work showed a good accuracy compared to prior, however, their accuracies for rare applications were bad which is a serious burden when applied to real-world scenarios. In addition, their models were designed with their network-specific features, such as IP addresses and port numbers, and hence can not be applied to the general internet classification. Later work used the same dataset with SVM and non-network-specific features to build models and reached an accuracy of 99% which is a vast enhancement compared to the earlier work Zhu, Yuan, and Guan (2007). Furthermore, a work presented in Williams, Zander, and Armitage (2006) used five different supervised machine learning algorithms to build multiple models and compare the performance of those different five algorithms in terms of complexity and accuracy. We urge the reader to refer to Dainotti et al. (2012) for a detailed survey of such different algorithms and datasets used for identifying the application from some flow data.

Supervised machine learning work mostly suffers from a common problem with datasets built for traffic identification which is the imbalance problem. In most real-time networks in which datasets are collected, some applications dominate the dataset and hence affect the accuracy in predicting other applications. Applications with a low number of training samples become religiously difficult to detect and hence end up in low clarification accuracy due to incorrect prediction Moore and Zuev (2005). The imbalance problem was discussed further in Provost (2008) and showed a critical problem in the use of traditional machine learning techniques for such applications. Different solutions were proposed in the literature to solve the imbalance problem. For example, In Charle, Rivera, Jesus, and Herrera (2015), the authors propose "artificial subclass sampling technology", which makes use of differences to generate a new class of positive samples between the positive class samples. This non-random sampling method in the data set artificially adds new information, much time will significantly improve the performance of the classification system. Of course, due to the added new sample may be wrong, at this time it will lead to a different effect of the subsequent classifier Han, Wang, and Mao (2005). Authors in Japkowicz and Stephen (2002) analyze the unbalanced nature of classification issues from four aspects including training sample size, the number of samples the negative and positive sample ratio, complexity of hyperplane and classification algorithms. In He and Garcia (2009), authors indicate the imbalanced distribution of the data is only a phenomenon; there may be two different reasons. The first possibility is imbalanced data owing to the underlying data causing uneven distribution, which is called the intrinsic imbalance. Another imbalanced data is not directly related to the nature of the data space, which is considered as an extrinsic imbalance. Extrinsic imbalances are equally as interesting as their intrinsic counterparts since the dataspace from which an extrinsic imbalanced data set is attained may not be imbalanced at all. In addition, Cieslak, Hoens, Chawla, and Kegelmeyer (2012), Brefeld and Scheffer (2005), Sun, Lim, and Liu (2010), Wang and Japkowicz (2010), Zieba, Tomczak, Lubicz, and Witek (2014), Zieba and Tomczak (2015) proposed different methodologies for solving the imbalance problem in different datasets.

2.2. Primarily definitions

To build datasets from the network traffic, one can collect payloads of different packets and analyze them to build machine learning models. However, such an approach suffers from computational power and storage requirements which are needed to analyze such information, especially in recent network paradigms. Generally, network hardware is not sufficient enough to store and process packets, and hence external storage units need to be used to save packet information. Alternatively,

Table 1
Proposed NETFLOW features.

Features	Features Description
Lport	low port number
Hport	high port number
duration	flow duration
transproto	flow transport protocol used (TCP / UDP)
TCPflags1	TCP header flag, or(OR), transport layer protocol is UDP, the feature is 0 (one direction)
TCPflags2	TCP header flag, or(OR), transport layer protocol is UDP, the feature is 0 (the other direction)
pps	packets/duration (second)
bps	bytes/duration (second)
mean packets arrived time	duration/packets
Bi-direction packets ratio	forward packets/ backward packets
Bi-direction bytes ratio	forward bytes/ backward bytes
Bi-direction packet length ratio	Bi-direction packets length ratio
Bi-direction packets	forward packets + backward packets
Bi-direction bytes	forward bytes + backward bytes
tos	bidirection tos OR from NETFLOW
mean packet length	Bi-direction bytes/bi-direction packets

we used NETFLOW information to get the information needed for each flow. Each flow record is composed of a sequence of network packets and an artificial logical equivalent to a call or connection [Rajahalme, Amante, Jiang, and Carpenter \(2011\)](#). [Definition 1](#) formally defines a NETFLOW flow record.

Definition 1. *NETFLOW flow records:*

A flow record X with t features can be represented as a vector of flow features $x_1, x_2, \dots, x_t$. Each of those features characterizes one property of a flow record. Examples of those features can be duration, port number, etc. A flow record can be represented as:

$$X = \{x_1, x_2, x_3, \dots, x_t\}$$

In our analysis, we used NETFLOW features that are presented in [Table 1](#). All the features are based on the packets exchanged between the clients and services. For example, *duration* describes the duration of a flow record. Applications like P2P use short-lived connections which results in a short duration while others like HTTP and SSH require higher duration. The packets per second (*pps*) feature describes the number of packets exchanged between the client and server. Video streaming applications usually cause a large number of packets exchange while other applications might have smaller packet exchange.

An application type or class is the type of application to be predicted by the machine learning models. Assuming that y is the application category, X is the vector of netflow features and F is the machine learning classifier. The prediction can be made as follows:

$$Y = F(X) = \{y_1, y_2, y_3, \dots, y_m\}$$

where $y_1, y_2, \dots, y_m$ represent different application categories, by function $F(X)$, traffic identification can be completed.

3. SVM and its extended versions

In this section, we briefly discuss traditional SVM and introduce weighted SVM, which extends SVM to solve the imbalance problem.

Furthermore, we discuss our proposed algorithm, CMSVM, which is proposed to solve the imbalance problem in a cost-effective solution.

3.1. SVM and weighted SVM algorithms

SVM is a well-known machine learning algorithm that builds hyperplanes to separate the data from multiple classes. To formulate the problem, let x be inputs from NETFLOW data, y is the list of targetted applications or class, then SVM would build a hyperplane H , which is defined as follows:

$$H : a^T \phi(x) + b = 0 \quad (1)$$

The assumption here is that the classes are linearly separable, a is a vector of the weights for each flow feature, b is the bias term and $\phi(\cdot)$ is fixed feature-space transformation. Assuming that x and y are known from prior data or the training dataset, the problem is to find the hyperplanes H that separate different classes. SVM can be formulated as the following optimization problem:

$$\min_{a,b} Q(a) = \frac{1}{2} a^T a + c \sum_{n=1}^N \quad (2)$$

$$s.t. y_n (a^T \phi(x_n) + b) \geq 1 - \varepsilon_n$$

where c controls the trade-off between the volume and errors, and $c > 0$, ε_n is the slack variable of the n th traffic object. Imbalanced classification is also an important problem in traffic classification and identification, while only the imbalanced classification problem of two-class SVM is discussed in biology and context classification, and the imbalanced classification problem of multi-class SVM is not discussed. So we will discuss the problem in this paper.

First, we introduce the two-class SVM classification method to resolve the imbalance problem which adopts the cost-sensitivity of classes from [Zieba et al. \(2014\)](#), parameter c is divided into two real-value parameters c_+ and c_- which represent the sets of positive and negative examples. Thus the optimization problem in Eq. (2) can be expressed as follows [Zieba et al. \(2014\)](#):

for all $n = 1, \dots, N$

$$\min_{a,b} Q(a) = \frac{1}{2} a^T a + c_+ \sum_{n \in N_+} \varepsilon_n + c_- \sum_{n \in N_-} \varepsilon_n \quad (3)$$

$$s.t. y_n (a^T \phi(x_n) + b) \geq 1 - \varepsilon_n$$

where $N_+ = n \in 1, \dots, N$ $y_n = 1$, and $N_- = n \in 1, \dots, N$ $y_n = -1$, w_n which is introduced as the penalty parameters variables. Eq. (3) is further defined as follows,

for all $n = 1, \dots, N$

$$\min_{a,b} Q(a) = \frac{1}{2} a^T a + c \sum_{n=1}^N \omega_n \varepsilon_n \quad (4)$$

$$s.t. y_n (a^T \phi(x_n) + b) \geq 1 - \varepsilon_n$$

for all $n = 1, \dots, N$

$$\sum_{n=1}^N \omega_n = N \quad (5)$$

$$\min Q(a) = \frac{1}{2} a^T a + c \sum_{n=1}^N \omega_n \varepsilon_n$$

where weights w_n meet the requirement of a probability distribution. Just like the paper Zieba et al. (2014) assume $w_n = \frac{1}{N}$, thus we will modify the two-class problem as a multi-class problem. So we give out $N_1, \dots, N_m$, where m indicates the type number of network traffic. Different type application has different weights because the different application has different proportion. Thus the optimization problem is further expressed as Eq. (6).

$$\begin{aligned} \min_{a,b} Q(a) &= \frac{1}{2}a^T a + \frac{CN^2}{2} \left(\frac{1}{N_1} \sum_{n=1}^N w_n \varepsilon_n + \frac{1}{N_2} \sum_{n=1}^N w_n \varepsilon_n + \dots + \frac{1}{N_t} \sum_{n=1}^N w_n \varepsilon_n \right) \\ &= \frac{1}{2}a^T a + \frac{CN^2}{2} \left(\frac{1}{N_1} + \frac{1}{N_2} + \dots + \frac{1}{N_m} \right) \sum_{n=1}^N w_n \varepsilon_n \\ &\quad s.t. y_n(a^T \phi(x_n) + b) \geq 1 - \varepsilon_n \\ &\quad \text{where } \text{all } n = 1, \dots, N, m = 1, \dots, T \end{aligned} \quad (6)$$

$$\max_{\lambda} q(\lambda) = \sum_{i=1}^N \lambda_i - \frac{1}{2} \sum_{i,j=1}^N \lambda_i \lambda_j y_i y_j k(x_i, x_j)$$

$$s.t. \sum_{i=1}^N \lambda_i y_i = 0 \quad (7)$$

$$0 \leq \lambda_n \leq C w_n$$

$$\text{for all } n = 1, \dots, N$$

where λ is the vector of Lagrange multipliers. k is the kernel function. Eq. (7) is detailed derived in the Appendix section. Finally, the identification model can be expressed as follows:

$$h(x_i) = \text{sign} \left(\sum_{n \in \text{SV}} y_n \lambda_n k(x_n, x_i) + b \right) \quad (8)$$

where sv shows the support vectors, the sign is the signum function which is an odd mathematical function that extracts the sign of a real number. $h(x_i)$ is the objective function.

3.2. CMSVM

In this section, we propose a cost-sensitive SVM based on multi-class with active learning, CMSVM, which can solve the imbalanced data by misclassification costs estimation and sampling selection. The process of CMSVM is the iterative procedure, which iteratively updates w by optimizing the exponential error Zieba et al. (2014). The value of error function e_i can be modified based on e_k in Zieba and Tomczak (2015), modified error function can be applied to the multi-class problem, which is calculated by the following:

$$e_i = \frac{E_l}{\frac{1}{N_1} \sum_{n=1}^N w_n^{(i)} + \frac{1}{N_2} \sum_{n=1}^N w_n^{(i)} + \dots + \frac{1}{N_t} \sum_{n=1}^N w_n^{(i)}} \quad (9)$$

where E_l is expressed by Eq. (10).

$$\begin{aligned} e_i &= \frac{1}{N_1} \sum_{n=1}^N w_n^{(i)} I(h_k(x_n) \neq y_n) + \frac{1}{N_2} \sum_{n=1}^N w_n^{(i)} I(h_k(x_n) \\ &\neq y_n) + \dots + \frac{1}{N_t} \sum_{n=1}^N w_n^{(i)} I(h_k(x_n) \neq y_n) \end{aligned} \quad (10)$$

where $I(\cdot)$ indicates the indicator function.

We consider weights w_n as misclassification cost, then the weight SVM will be changed as the cost-sensitive SVM, in order to improve the training speed, reduce the number of training samples and time. We introduce the active learning mechanism into cost-sensitive multi-class SVM and call it CMSVM which includes two parts, one is cost-sensitive SVM, the other is to select the optimal new sample to train by query strategy. In this paper, Assume labeled network flow as L and unlabeled network flow as U , we can adopt the query strategy to select the new samples from U to the label.

Fig. 1 shows the procedure of CMSVM. Firstly, labeled network flow (L) and unlabeled network flow (U) are considered as the input samples. We train the cost-sensitive SVM adopting L data, then classify all U and judge the maximum iteration number, if it meets the requirement for the CMSVM algorithm, the algorithm should end. Or else we select the network flow samples close to the cost-sensitive SVM hyperplane which

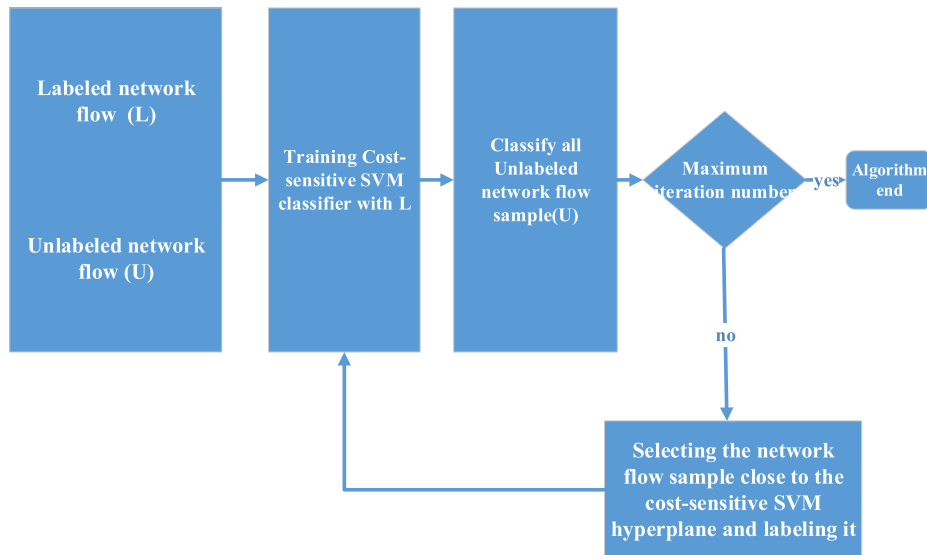


Fig. 1. CMSVM flow chart.

Table 2
MOORE_SET Dataset.

AppID	Category	Application	Number of flows	Proportion
1	WWW	HTTP, HTTPS	328,091	86.91
2	BULK	FTP	11,539	3.056
3	MAIL	POP3, IMAP, SMTP	28,567	7.567
4	DB	SQLnet, Oracle	2648	0.701
5	SERV	DNS, NTP, LDAP	2099	0.556
6	P2P	Kazaa, Bittorrent, Gnutella	2094	0.555
7	ATTACK	Worm and virus Attacks	1793	0.475
8	MULT	Media Player, Real	1152	0.305
9	INT	SSH, klogin, Telnet	110	0.029
10	GAME	Halflife	8	0.002

is a query strategy and label it. Those samples are inserted training cost-sensitive SVM to generate the new classifier.

Algorithm: CMSVM

Algorithm 1: CMSVM algorithm

```

Input: Flow F include all features(L and U), I: Iteration Number
Output: Class label and Classifier for imbalance data  $H(x) = \arg\max_{y \in Y} \sum_{k=1}^{K_{final}} c_k I(h_k(x) = y)$ 
1 Begin
2 Initialize flow feature;
3 Generate new Flow F;
4 for k = 1 to I
5 Train cost-sensitive SVM by solving(6) with values of  $w_n^{(k)}, N_1^{(k)}, N_2^{(k)}, \dots, N_t^{(k)}$ ;
6 while  $e_k < 0.5$  do
7  $c_k \leftarrow \ln \frac{1 - e_k}{e_k}$ 
8 compute Gmean value  $g_k$  and Mauc value  $m_k$ ;
9  $H_k(x) = \arg\max_{y \in Y} \sum_{i=1}^k c_i I(h_i(x) = y)$ 
10 if  $g_k > G$  then
11  $G \leftarrow g_k$ , Select the network flow which is close to the cssvm hyperplane in cssvm;
12  $K_{final} \leftarrow k$ ;
13 end if
14 Return  $H$ ;
15 end while
16 End

```

Algorithm1 represents a cost-sensitive SVM method based on multi-class with active learning, the first step initializes network flow features and generate new flow, those network flow samples include labeled network flow(L) and unlabeled network flow(U). Next, the cost-sensitive SVM classifier is built, and the classifier is increasingly optimal by e_k . Gmean value and Mauc value are computed. When the error value is lower than 0.5, the value c_k will be further computed. Finally, the classifier is updated in the iteration procedure.

3.3. Algorithm complexity analysis

In this section, the computational complexity of CMSVM is analyzed. The complexity depends on three parts. The first part is the initialization process of flow feature with the complexity of $O(1)$. The second part is the training phase with the complexity of $O(I)$. The final part is computing process of Gmean and Mauc value with the compexity of $O(1)$. So the total computational complexity of CMSVM is $2O(1) + O(I)$ and approximately equal to $O(I)$.

4. Datasets and performance metrics

In this section, we describe the datasets used to evaluate the proposed algorithm. In addition, we discuss the performance metrics that

Table 3
NOC_SET Dataset.

AppID	Application	Protocol	Flow number	Proportion
1	WWW	HTTP	4943	64.6
2	Bulk	FTP	39	0.5
3	Mail	IMAP, POP3, SMTP	91	1.19
4	P2P	BitTorrent, eDonkey, XunLei, etc.	1414	18.5
5	Service	DNS, NTP	433	5.7
6	Interactive	SSH, CVS, pc Anywhere	6	0.08
7	Multimedia	RTSP, Real	20	0.3
8	Voice	SIP, Skype	276	3.6
9	Others	games, attacks	431	5.6

Table 4
Confusion matrix.

	Predicted positive class	Predicted negative class
Actual positive class	TP	FN
Actual negative class	FP	TN

will be used to prove the effectiveness of the machine learning models to correctly predict the class of a flow and solve the imbalance problem.

4.1. Dataset

In this work, we used two datasets, namely MOORE_SET, [Moore and Papagiannaki \(2005\)](#) dataset and NOC_SET dataset, [Shi, Zhou, Wei et al. \(2013\)](#) to build and test models using different machine learning algorithms. Those datasets are publicly available datasets that can be summarized as follows: *MOORE-SET*: as originally given by [Moore and Papagiannaki \(2005\)](#) MOORE-SET dataset generates flow data using random sampling over several different periods from one node on the internet. It has 10 applications with a different number of samples as summarized in [Table 2](#). This network traffic collection site is shared by about 1000 researchers, technicians, and management staff of three research institutions, and connected to the Internet through a full-duplex Gigabit Ethernet link. All full-duplex traffic on this connection is captured in a full 24 h period, so the original traffic-set contained all full-duplex traffic connected to the node in both link directions. Since the original traffic-set is too large, Moore divides it into ten subsets by a random sampling method. The sampling times of each subset are almost the same (approximately 1680 s each) and the non-overlapping random samples are uniformly distributed over the 24-hour interval. The number of flows and the proportion of the various types of network traffic are shown in [Table 2](#).

NOC-SET: NOC_SET [Shi, Zhou, Wei et al. \(2013\)](#) dataset is collected at Southeast University and each flow is labeled by modified L7_filter [Levandoski \(2008\)](#) software. It adopts packets collector to obtain 200G packets data in 12 h. To suit for generating the flow label, we modified the L7_filter software to label the packets and group the packets labeled into flows. A TCP/IP flow is uniquely identified by the five values: source IP, destination IP, source port, destination port, and protocol type. For TCP flow, we select semantically complete TCP flows in our experiments, which means that each TCP flow contains successful three-way handshake packets and connection termination (FIN-ACK) packets. [Table 3](#) summarizes the flow information of NOC_SET dataset. There are nine application classes, some of which contain multiple application layer protocols.

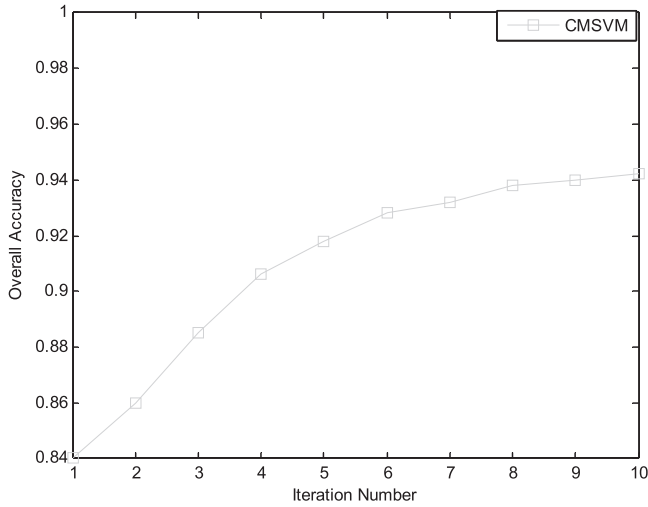


Fig. 2. Overall accuracy results for CMSVM algorithms with MOORE_SET.

4.2. Performance metrics

To evaluate the effectiveness of the proposed algorithm, we use different standard evaluation metrics that are used in literature to evaluate any learning algorithm. Namely, we calculate the overall accuracy, precision and recall to check the correctness in predicting a traffic flow in a particular class. In addition, to evaluate the effectiveness of algorithms in solving the imbalance problem, we used the geometric mean (Gmean) and the multi-class area under the curve (MAUC) Sun et al. (2010) which are commonly used metrics to evaluate the solution to the imbalance problem. Gmean is a type of mean or average, which indicates the central tendency or typical value of a set of numbers by using the product of their values. While MAUC is improved AUC method to distinguish between multi classes.

True positives (TP) represent flows in class A that are classified correctly in that class. False negatives (FP) represent flows that are not in class A but misclassified in that class. False negatives (FN) represent flows in class A that are misclassified as some other category; in other words, not detected in their class. FN will result in identification accuracy loss. True negatives (TN) represent flows that are not in class A and correctly not detected in class A. We can obtain the confusion matrix presented in Table 4 after a classification process. Then, the calculation of our metrics goes as follows:

Precision: The percentage of samples classified in a particular class correctly compared to all samples classified in that class.

$$\text{Precision} = \frac{TP}{TP + FP} \quad (11)$$

Recall: The percentage of samples in a particular class A that are correctly classified in that class

$$\text{Recall} = \text{TPR} = \frac{TP}{TP + FN} \quad (12)$$

The TN rate TNR or specificity Sp is defined as

$$\text{TNR} = \frac{TN}{TN + FP} \quad (13)$$

Overall accuracy: The percentage of samples that are correctly classified

$$\text{OverallAccuracy} = \frac{\sum_{i=1}^n TP_i}{\sum_{i=1}^n TP_i + FP_i} \quad (14)$$

where n is where n is the number of samples.

G-mean: it is called the geometric mean.

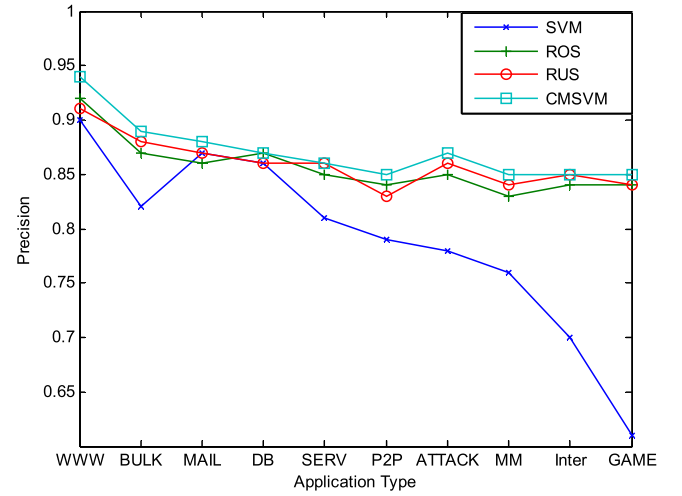


Fig. 3. Precision results for different applications with the different used algorithms with MOORE_SET.

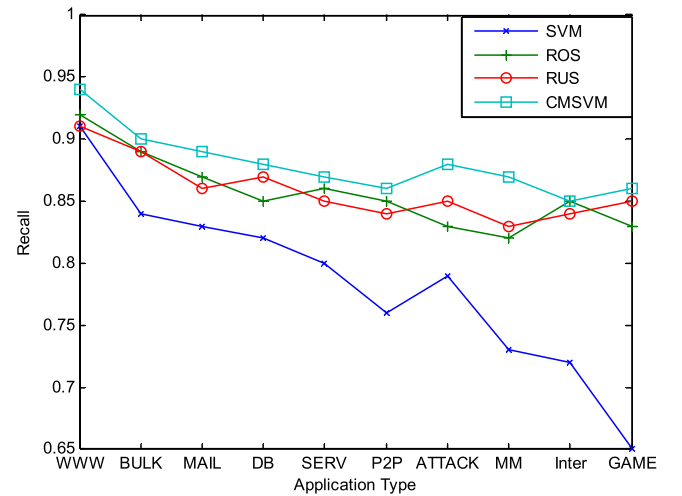


Fig. 4. Recall results for different applications with the different used algorithms with MOORE_SET.

$$Gmean = \sqrt{TPR * TNR} \quad (15)$$

where n is the number of samples.

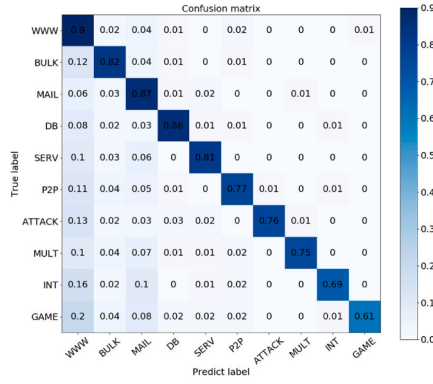
MAUC: area under the curve (AUC) is a standard measure for a single class dataset where you distinguish between only two classes. However, such metric can not be applied to a multi-class dataset where more than two class is involved. Hence, the authors in Sun et al. (2010) proposed to transform AUC to MAUC in order to be applied to multi-class datasets. MAUC can be defined as follows:

$$Mauc = \frac{2}{q(q-1)} \sum_{i < j} AUC(c_i, c_j) \quad (16)$$

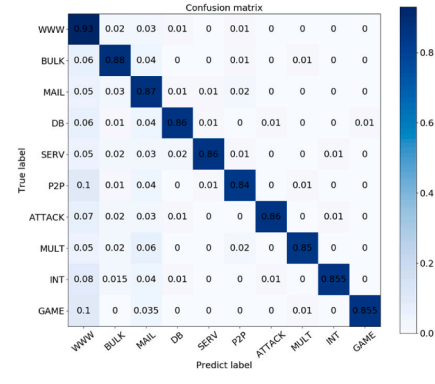
$$\text{Where } AUC(c_i, c_j) = \frac{AUC(c_i/c_j) + AUC(c_j/c_i)}{2}, AUC(c_i/c_j) = \frac{S_i - n_{c_i}(n_{c_i} + 1)/2}{n_{c_i}n_{c_j}}$$

5. Experimental results

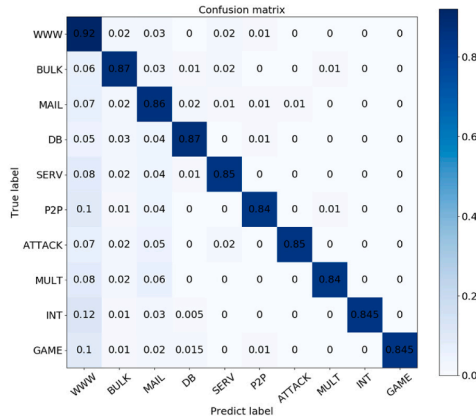
In this section, we describe the performance evaluation of our algorithm, CMSVM, compared to different other algorithms that are designed to solve the imbalance problem. More specifically, we compare CMSVM to SVM, ROS, and RUC using recall, precision,



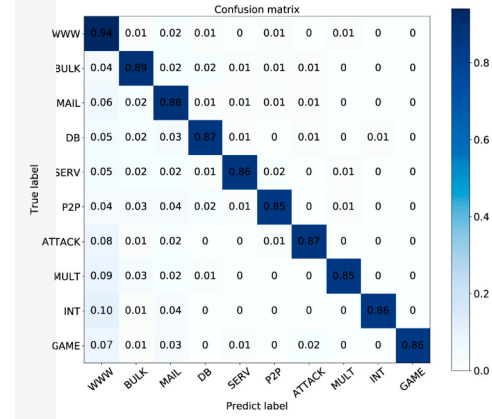
(a) SVM



(b) ROS



(c) RUS



(d) CMSVM

Fig. 5. Results for confusion matrix with the different used algorithms with MOORE_SET.

Table 5
Experiment results for MOORE_SET.

Method	Gmean	Mauc
SVM	0.583	0.625
ROS	0.673	0.631
RUS	0.684	0.646
CMSVM	0.718	0.709

overall accuracy, Gmean, and MAUC performance metrics. We used the WEKA Hall et al. (2009) machine learning suite to build an SVM model. As discussed earlier, two datasets, namely MOORE_SET and NOC_SET, are used to verify our models and calculate the performance metrics from the results on those two datasets. Tenfold cross-validation was used to cross-validate the data. The ten-fold cross-validation method is a commonly used accuracy test method. Its basic idea is to divide the data set into 10 parts, and take 9 parts as training data and 1 part as test data, in turn, to carry out experiments. Each experiment will get the corresponding accuracy, and the average of the accuracy of 10 results is used as the estimation of the accuracy of the algorithm. Experimental results are shown in Figs. 2–5, and Table 5 for MOORE_SET while they are shown in Figs. 6–9, and Table 6 for NOC_SET.

5.1. Performance analysis with MOORE_SET

Fig. 2 represents the overall accuracy for CMSVM in different iteration number. We find the CMSVM algorithm gets convergence and

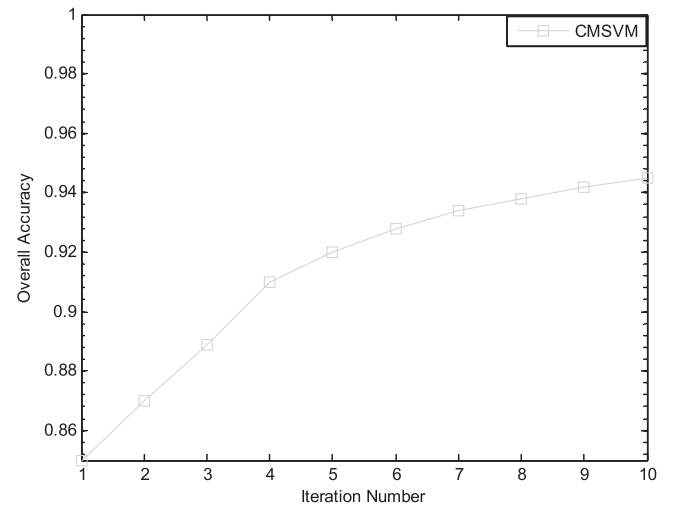


Fig. 6. Overall accuracy results for CMSVM algorithms with NOC_SET.

overall accuracy reaches 0.942 which is the maximum. So we will adopt the classifier to classify the network flow when the iteration number is 10. As can be seen in Figs. 3 and 4, CMSVM, ROS, and ROS had a high precision and recall with all applications whereas SVM had a relatively bad recall and precision for most of the applications. Thus, SVM did not perform well due to the imbalance problem as applications had a different ratio of the number of samples. As discussed in

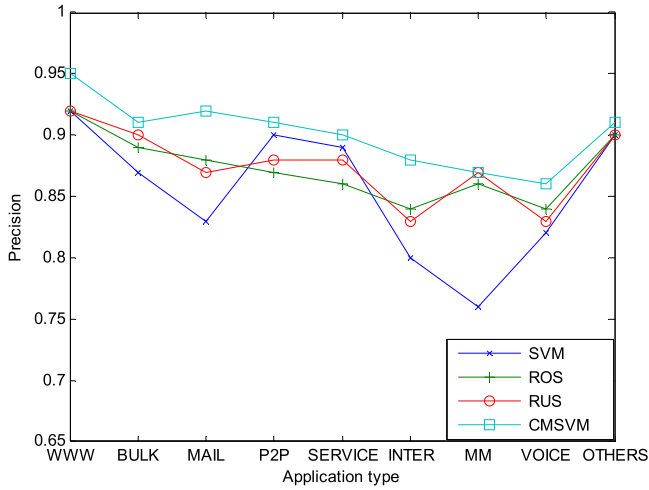


Fig. 7. Precision results for different applications with the different used algorithms with NOC_SET.

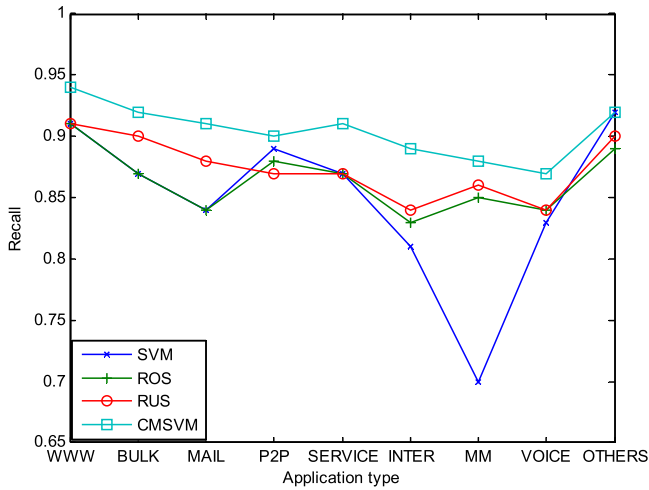


Fig. 8. Recall results for different applications with the different used algorithms with NOC_SET.

Section 4.1, the Game application had a low number of samples and hence traditional SVM can not predict it. However, other algorithms performed well with all applications and CMSVM had the most stable and highest performance compared to the other algorithms. Fig. 5 presents the confusion matrix of four algorithms. It is observed that network traffic with a large proportion has a high TP value in four algorithms. However, there is a big difference on TP for network traffic with a small proportion. In Fig. 5(a), the SVM has 0.61 TP value on game category, ROS, RUS and CMSVM has 0.855, 0.845 and 0.86 TP value. The results indicate that adopting the unbalanced strategy can improve the identification accuracy of network traffic with a small proportion. Especially, our proposed CMSVM has 0.86 TP value. For the network traffic www with a large proportion, it can achieve 0.94 TP value. Moreover, MAUC and Gmean results, as presented in Table 5 showed the outperformance of CMSVM compared to other algorithms. CMSVM can reach 72% Gmean value while others had lower values which can reach 58% with SVM. In addition, CMSVM reaches around 70% MAUC while others reach 65% by maximum which shows the better performance of our proposed algorithm compared to the others.

5.2. Performance analysis with NOC_SET

Fig. 6 represents the overall accuracy for CMSVM in different iteration number. We find the CMSVM algorithm gets convergence and overall accuracy reaches 0.945 which is the maximum. So we will adopt the classifier to classify the network flow when the iteration number is 10. NOC_SET results are shown in Figs. 7 and 8 and Table 5 for precision, recall, and Gmean and MAUC. As can be seen, CMSVM outperforms all other algorithms in terms of precision and recall. Similar to MOORE_SET, SVM had the worst performance for a low number of sample applications which can be reasoned by the inability of SVM to solve the imbalance problem. On the other hand, CMSVM had a slightly better performance than ROS and RUS with more stable performance which proof their efficiency in predicting even a low number of sample applications. Results for the confusion matrix with the different used algorithms are depicted in Fig. 9. From Table 3, we know the network traffic with minimum proportion is Interactive, which only is 0.08. In Fig. 9(a), SVM algorithm's TP value is 0.81. From Fig. 9(b), (c), and (d), we can know ROS, RUS, and CMSVM algorithm's TP value is 0.835, 0.84, and 0.89. The experimental results show our proposed CMSVM algorithm can better improve the identification performance. For Gmean and MAUC, as shown in Table 6, CMSVM performed better than other algorithms and could reach about 77% Gmean and 75% MAUC. This can be due to the better learning ability of CMSVM using cost-sensitive than applying any of the resamplings.

To summarize this section, our proposed algorithm, CMSVM, could achieve higher precisions and recall compared to the other three methods. It showed a better performance in Gmean and MAUC which is effective proof in solving the imbalance problem. By adding other features to the features mentioned in Table 1, better classification results can be achieved with better models and reasonably added complexity. In addition, the proposed multi-class solution can be applied to other machine learning algorithms to improve their efficiency in solving the imbalance problem.

6. Conclusions

Traffic identification is one of the critical issues for network traffic planning and management. While methods based on machine learning are getting popularity in solving such issues, imbalance problems in existing datasets is a burden against the efficiency of those algorithms. This paper proposed an improved SVM algorithm, CMSVM, which tries to solve the imbalance problem by introducing the weight values and active learning for each class of traffic. MOORE_SET and NOC_SET datasets are used to build models of different learning algorithms and compare their performances. SVM, ROS, and RUS algorithms are used in performance analysis to show the efficiency of the proposed algorithm when compared to the other three algorithms. Experimental results showed that CMSVM can outperform others in terms of recall, precision, overall accuracy, MAUC, and Gmean. However, CMSVM still gets about 70% MAUC and Gmean which implies that it still needs improvements. Adding more features to the dataset or applying the proposed strategy to other machine learning algorithms can improve the performance and results in better solutions for the imbalance problem. In the future work, we will consider the joint optimization of deep learning and migration strategy with imbalance problem.

Ethical approval

This article does not contain any studies with human participants or animals performed by any of the authors.

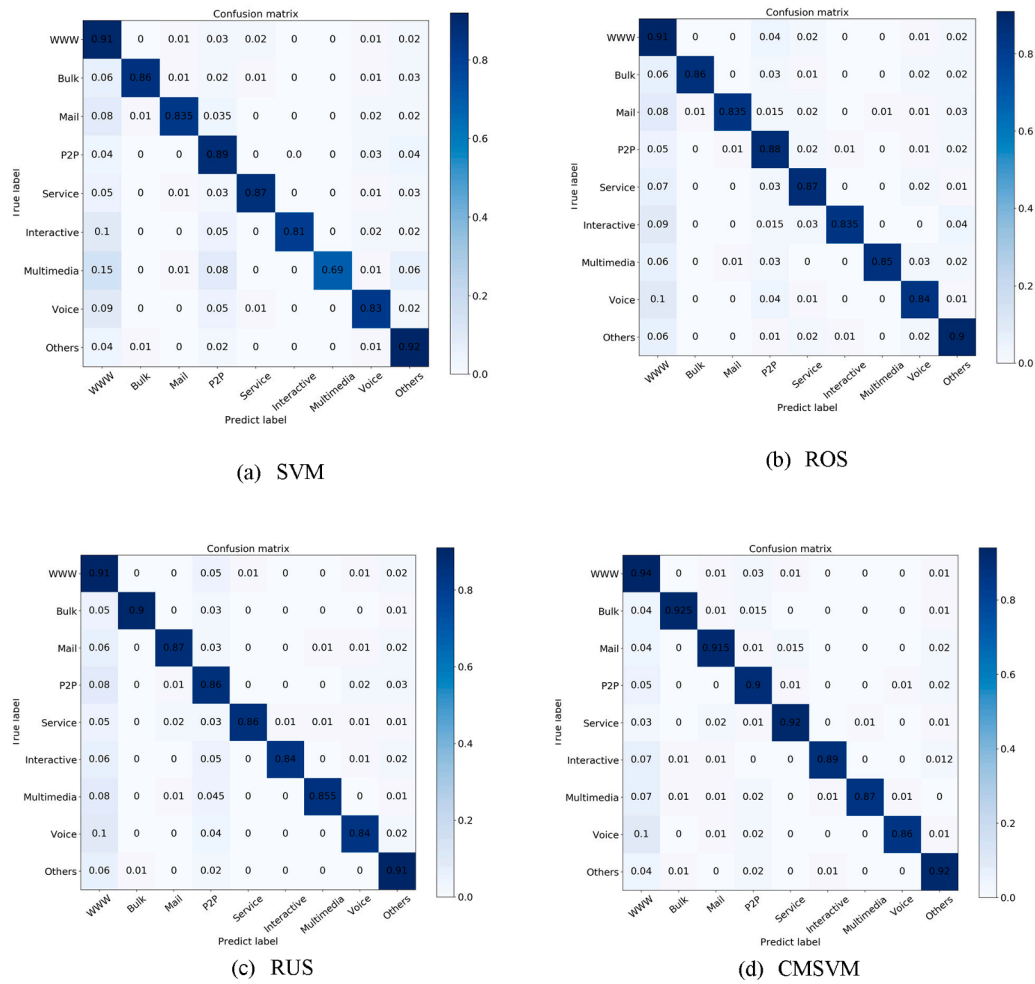


Fig. 9. Results for confusion matrix with the different used algorithms with NOC_SET.

Table 6

Experiment results for NOC_SET.

Method	Gmean	Mauc
SVM	0.674	0.71125
ROS	0.723	0.7231
RUS	0.734	0.7256
CMSVM	0.768	0.774

CRedit authorship contribution statement

Shi Dong: Conceptualization, Methodology, Software, Writing - original draft, Writing - review & editing.

Appendix A

The prime problem is:

$$\min_{a,b} Q(a) = \frac{1}{2}a^T a + C_+ \sum_{n \in I'_+} \xi_n + C_- \sum_{n \in I'_-} \xi_n$$

$$s.t. y_n(a^T \phi(x_n) + b) \geq 1 - \xi_n$$

for all $n = 1, \dots, N$

Declaration of Competing Interest

The all authors of the paper immediately below certify that they have no affiliations with or involvement in any organization or entity with any financial interest.

Acknowledgments

This paper is supported by Project supported by Key Scientific and Technological Research Projects in Henan Province (Grand No. 192102210125), Open Foundation of State key Laboratory of Networking and Switching Technology (Beijing University of Posts and Telecommunications) (KLNST-2020-2-01). In addition, the authors also will thank the anonymous reviewers for their comments and suggestions.

(17)

where $\mathcal{N}_+ = \{n \in \{1, \dots, N\} : y_n = 1\}$, and $\mathcal{N}_- = \{n \in \{1, \dots, N\} : y_n = -1\}$.

Further, we generalize the problem 1 by introducing individual penalty parameters w_n for each n th observation which yields:

$$\begin{aligned} \min_{a,b} Q(a) &= \frac{1}{2} a^T a + C \sum_{n=1}^n w_n \xi_n \\ \text{s.t. } y_n (a^T \phi(\mathbf{x}_n) + b) &\geq 1 - \xi_n \end{aligned} \quad (18)$$

for all $n = 1, \dots, N$

Because we have maintained the individual penalty parameter C , the penalty parameters w_n have to fulfill the following condition:

$$\sum_{n=1}^N w_n = N \quad (19)$$

The introduction of w allows to diversify the costs of misclassification not only between classes but also within classes. Further, we will use the penalty parameters w in the boosting procedure during the construction of the classifier. Then we want to get the dual optimization problem of the primal optimization problem 2.

Introduce the multipliers λ we get the Lagrange function:

$$\begin{aligned} L(a, b, \xi, \lambda) &= \frac{1}{2} a^T a + C \sum_{n=1}^n w_n \xi_n + \lambda_1 (y_1 (a^T \phi(\mathbf{x}_1) + b) + 1 - \xi_1) \\ &\quad + \lambda_2 (y_2 (a^T \phi(\mathbf{x}_2) + b) + 1 - \xi_2) + \dots + \lambda_N (y_N (a^T \phi(\mathbf{x}_N) + b) + 1 - \xi_N) \end{aligned} \quad (20)$$

Where $\lambda = (\lambda_1, \lambda_2, \dots, \lambda_N)$ and $\xi = (\xi_1, \xi_2, \dots, \xi_N)$. Let $\frac{\partial L}{\partial a}$, $\frac{\partial L}{\partial b}$ and $\frac{\partial L}{\partial \xi_i}$ equal to 0, we get

$$\frac{\partial L}{\partial a} = a - \lambda_1 y_1 \phi(\mathbf{x}_1) - \lambda_2 y_2 \phi(\mathbf{x}_2) - \dots - \lambda_N y_N \phi(\mathbf{x}_N) = 0 \quad (21)$$

$$\frac{\partial L}{\partial b} = \lambda_1 y_1 + \lambda_2 y_2 + \dots + \lambda_N y_N = 0 \quad (22)$$

$$\frac{\partial L}{\partial \xi_i} = C w_i - \lambda_i = 0 \quad \text{for all } i = 1, 2, \dots, N \quad (23)$$

From Eq. (21) we have

$$a = \sum_{i=1}^N \lambda_i y_i \phi(\mathbf{x}_i) \quad (24)$$

Put Eq. (24) into (20), we have the dual function

$$\begin{aligned} q(\lambda) &= \inf_{a,b,\xi} L(a, b, \xi, \lambda) \\ &= \frac{1}{2} \sum_{i=1}^N \lambda_i y_i \phi(\mathbf{x}_i)^T \sum_{i=1}^N \lambda_i y_i \phi(\mathbf{x}_i) + C \sum_{n=1}^n w_n \xi_n + \lambda_1 (y_1 (\sum_{i=1}^N \lambda_i y_i \phi(\mathbf{x}_i)^T \phi(\mathbf{x}_1) + b) + 1 - \xi_1) \\ &\quad + \lambda_2 (y_2 (\sum_{i=1}^N \lambda_i y_i \phi(\mathbf{x}_i)^T \phi(\mathbf{x}_2) + b) + 1 - \xi_2) + \dots + \lambda_N (y_N (\sum_{i=1}^N \lambda_i y_i \phi(\mathbf{x}_i)^T \phi(\mathbf{x}_N) + b) + 1 - \xi_N) \\ &= -\frac{1}{2} \sum_{i=1}^N \lambda_i y_i \phi(\mathbf{x}_i)^T \sum_{i=1}^N \lambda_i y_i \phi(\mathbf{x}_i) + C \sum_{i=1}^i w_i \xi_n + \sum_{i=1}^N \lambda_i - \sum_{i=1}^N \lambda_i y_i b + \sum_{i=1}^N \lambda_i \xi_i \end{aligned} \quad (25)$$

According to Eqs. (22) and (23), we have

$$\begin{aligned} q(\lambda) &= \sum_{i=1}^N \lambda_i - \frac{1}{2} \sum_{i,j=1}^N \lambda_i \lambda_j y_i y_j \phi(\mathbf{x}_i)^T \phi(\mathbf{x}_j) \\ &= \sum_{i=1}^N \lambda_i - \frac{1}{2} \sum_{i,j=1}^N \lambda_i \lambda_j y_i y_j k(\mathbf{x}_i, \mathbf{x}_j) \end{aligned} \quad (26)$$

where $k(\mathbf{x}_i, \mathbf{x}_j)$ is a kernel function. Therefore, we get the following dual optimization problem:

$$\max_{\lambda} q(\lambda) = \sum_{i=1}^N \lambda_i - \frac{1}{2} \sum_{i,j=1}^N \lambda_i \lambda_j y_i y_j k(\mathbf{x}_i, \mathbf{x}_j) \quad (27)$$

$$\text{s.t. } \sum_{i=1}^N \lambda_i y_i = 0 \quad (28)$$

$$0 \leq \lambda_n \leq C w_n \quad (29)$$

$$\text{for all } n = 1, \dots, N \quad (30)$$

References

- Aceto, G., Ciunzo, D., Montieri, A., & Pescapé, A. (2018). Multi-classification approaches for classifying mobile app traffic. *Journal of Network and Computer Applications*, 131–145.
- Aceto, G., Ciunzo, D., Montieri, A., & Pescapé, A. (2019a). MIMETIC: Mobile encrypted traffic classification using multimodal deep learning. *Computer Networks*, 165 (Dec.24), 106944.1–106944.12.
- Aceto, G., Ciunzo, D., Montieri, A., & Pescapé, A. (2019b). Mobile encrypted traffic classification using deep learning: Experimental evaluation, lessons learned, and challenges. *IEEE Transactions on Network and Service Management*, 445–458.
- Brefeld, U., & Scheffer, T. (2005). AUC maximizing support vector learning. *ICML Workshop on Roc Analysis in Machine Learning*, 377–380.
- Buda, M., Maki, A., & Mazurowski, M. A. (2018). A systematic study of the class imbalance problem in convolutional neural networks. *Neural Networks*, 106, 249–259.
- Charte, F., Rivera, A. J., Jesus, M. J. D., & Herrera, F. (2015). Addressing imbalance in multilabel classification: Measures and random resampling algorithms. *Neurocomputing*, 163, 3–16.
- Cieslak, D. A., Hoens, T. R., Chawla, N. V., & Kegelmeyer, W. P. (2012). Hellingier distancedecisiontreesarerobustandskew-insensitive. *Data Mining Knowledge Discovery*, 24(1), 136–158.
- Dainotti, A., Pescapé, A., & Claffy, K. C. (2012). Issues and future directions in traffic classification. *Network IEEE*, 26(1), 35–40.
- Hall, M., Frank, E., Holmes, G., Pfahringer, B., Reutemann, P., & Witten, I. H. (2009). The weka data mining software: An update. *ACM SIGKDD Explorations Newsletter*, 11(1), 10–18.
- Han, H., Wang, W. Y., & Mao, B. H. (2005). Borderline-smote: A new oversampling method in imbalanced data sets learning. *Lecture Notes in Computer Science*, 3644(5), 878–887.
- He, H., & Garcia, E. A. (2009). Learning from imbalanced data. *IEEE Transactions on Knowledge Data Engineering*, 21(9), 1263–1284.
- Japkowicz, N., & Stephen, S. (2002). The class imbalance problem: A systematic study. *Intelligent Data Analysis*, 6(5), 429–449.
- Kumar, S., Turner, J., & Williams, J. (2006). Advanced algorithms for fast and scalable deep packet inspection. *Architecture for Networking Communications Systems, Acm/IEEE Symposium*.
- Levandoski, J., 2008. Application layer packet classifier for linux. <http://17-filter.sourceforge.net/>.
- Montieri, A., Ciunzo, D., Aceto, G., & Pescapé, A. (2017). Anonymity Services Tor, I2P, JonDonym: Classifying in the Dark. In *29th International Teletraffic Congress (ITC 29) Genoa* (pp. 81–89). <https://doi.org/10.23919/ITC.2017.8064342>
- Moore, A. W., & Papagiannaki, K. (2005). Toward the accurate identification of network applications. *International Conference on Passive Active Network Measurement*.
- Moore, A. W., & Zuev, D. (2005). Internet traffic classification using bayesian analysis techniques. *Acm Sigmetrics Performance Evaluation Review*, 33(1), 50.
- Nguyen, T. T. T., & Armitage, G. J. (2009). A survey of techniques for internet traffic classification using machine learning. *IEEE Communications Surveys Tutorials*, 10(4), 56–76.
- Provost, F. (2008). Machine learning from imbalanced data sets 101 (extended abstract). *Soft Computing Pattern Recognition, International Conference of*.
- Rajahalme, J., Amante, S., Jiang, S., Carpenter, B., 2011. Ipv6 flow label specification.
- Shi, D., Zhou, D., Wei, D., & Jian, G. (2013). Flow cluster algorithm based on improved k-means method. *Iete Journal of Research*, 59(4), 326–333.
- Shi, D., Zhou, D. D., Zhou, W., Wei, D., & Jian, G. (2013). Research on network traffic identification based on improved bp neural network. *Applied Mathematics Information Sciences*, 7(1), 389–398.
- Sun, A., Lim, E. P., & Liu, Y. (2010). On strategies for imbalanced text classification using svm: A comparative study. *Decision Support Systems*, 48(1), 191–201.
- Wang, B. X., & Japkowicz, N. (2010). Boosting support vector machines for imbalanced data sets. *Knowledge Information Systems*, 25(1), 1–20.
- Williams, N., Zander, S., & Armitage, G. J. (2006). A preliminary performance comparison of five machine learning algorithms for practical ip traffic flow classification. *Acm Sigcomm Computer Communication Review*, 36(5), 5–16.
- Zhu, L., Yuan, R., & Guan, X. (2007). Accurate classification of the internet traffic based on the svm method. *IEEE International Conference on Communications*.
- Zieba, M., & Tomczak, J. M. (2015). Boosted svm with active learning strategy for imbalanced data. *Soft Computing*, 19(12), 3357–3368.
- Zieba, M., Tomczak, J. M., Lubicz, M., & Witek, J. (2014). Boosted svm for extracting rules from imbalanced data in application to prediction of the post operative life expectancy in the lung cancer patients. *Applied Soft Computing Journal*, 14(1), 99–108.



Shi Dong received M.S in computer science from University of Electronic and Technology of China in 2009, Ph.D in computer science from southeast university in 2013. He worked as post doctor researcher in Huazhong University of Science and Technology. He is a visiting scholar in Washington University in St.Louis. He is currently a Distinguished Professor with Zhoukou Normal University. His current research interests include network management and network security.